



Web Technologies

WET2VO

Teil 5: Komponenten und Standards

Medientechnik und -design | SS 2026

Komponenten

Eine Motivation



PHP zu Beginn: DIY

Frame- works



Ab Mitte der 2000er: Komplettpaket für die Webentwicklung
Zend Framework, Symfony, Code Igniter

Komponenten

Eine Sammlung von PHP-Code, die
genau ein Problem löst.





Kriterien für gute Komponenten

Fokussiert auf ein Problem

Erfüllt **genau eine Aufgabe**, daher wenig Overhead.

Nur **so viel Code wie nötig**, um die Aufgabe zu erfüllen.

Kooperativ

Komponenten **arbeiten gut** mit anderen Komponenten **zusammen**.

Gut getestet

Tests vermeiden Probleme und sind Nachvollziehbar.

Gut dokumentiert

Die Verwendung der Komponente ist genau **dokumentiert**, ebenso der Quellcode.



Komponenten **vs.** Frameworks

Komponenten

HTTP-Client Library, E-Mail-Library,
Dateisystem-Abstraktionsschicht,
Image-Manipulation,
Datenbankzugriff etc.

Frameworks

Symfony, Laravel, Yii, Laminas,
CakePHP, Nette, Phalcon, Spiral etc.

Komponenten verwalten

Composer & Packagist



Composer : Dependency- Manager

Installiert und verwaltet Packages
und deren Abhängigkeiten auf
Projektbasis.
Übernimmt das Autoloading der
Komponenten.

<https://getcomposer.org/>



Composer Installation



Installation unter Linux/Unix/Mac OS

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php -r "if (hash_file('sha384', 'composer-setup.php') ===
'c8b085408188070d5f52bcfe4ecfbee5f727afa458b2573b8eaaf77b3419b0bf2768dc67c86944da1544f06fa544fd47') {
echo 'Installer verified'.PHP_EOL; } else { echo 'Installer corrupt'.PHP_EOL;
unlink('composer-setup.php'); exit(1); }"
```

`php composer-setup.php`

```
php -r "unlink('composer-setup.php');"
```

```
mv composer.phar /usr/local/bin/composer
```

Im fhoove-web-dock
Docker-Container bereits
installiert!

Installation unter Windows (lokale PHP-Installation):

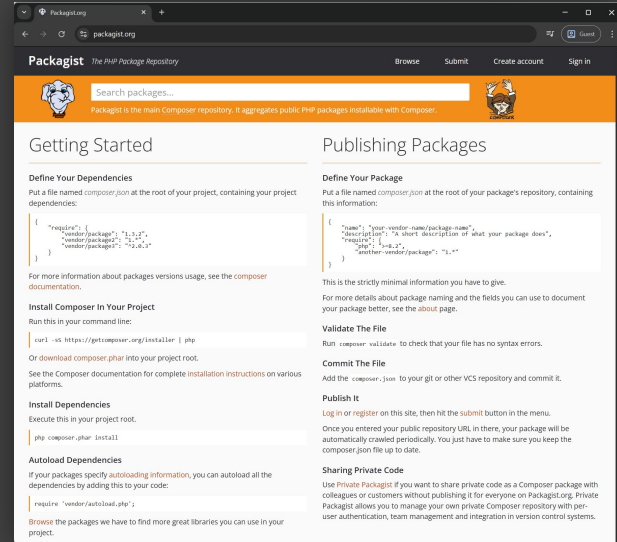
Composer-Setup.exe



Packagist : Package Repository

Zentrales Verzeichnis zum Finden
und Abrufen von
PHP-Komponenten mit Hilfe von
Composer.

<https://packagist.org/>



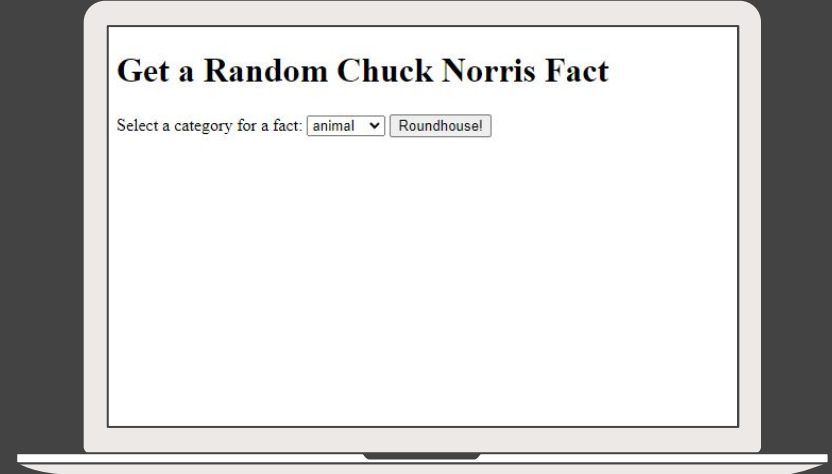
Anzahl Komponenten:
451.631

Kuratierte Liste:
<https://github.com/ziadoz/awesome-php>

Beispiel : Random Chuck Norris Fact

Die verfügbaren Kategorien an Facts von der [Chuck Norris Jokes Api](#) abfragen.

Die Kategorien in einer Dropdown-Liste anzeigen und nach Auswahl einen zufälligen Fakt aus dieser Kategorie ausgeben.



Beispiel: Requirements



Idee

PHP-Skript wird aufgerufen und setzt zunächst einen Request an die Chuck Norris Jokes Api ab (PHP-Skript wird selbst zum Client). <https://api.chucknorris.io/jokes/categories> liefert alle Kategorien, in denen Facts vorhanden sind. Mit den Kategorien wird eine Dropdown-Liste befüllt. User*innen wählen eine Kategorie und schicken das Formular ab.

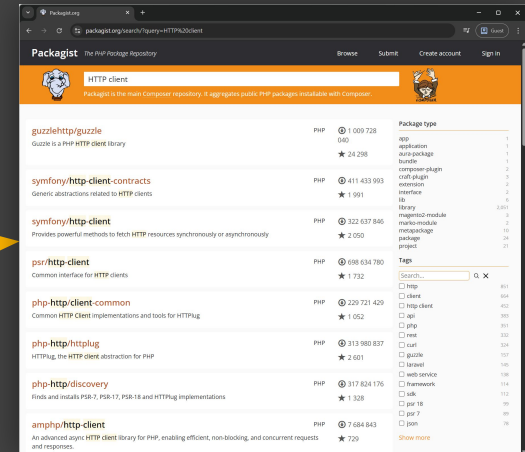
Ein zweiter Request wird an <https://api.chucknorris.io/jokes/random> geschickt, Ein Get-Parameter **category** ermöglicht es, eine Kategorie zu übergeben: <https://api.chucknorris.io/jokes/random?category=science>

Benötigt

HTTP-Client-Library
um einen Request an api.chucknorris.io abzusetzen.

Vorgehensweise: Suche auf Packagist nach **HTTP client**

Ergebnis: `guzzlehttp/guzzle`





Komponentennamen

vendor/package

guzzlehttp/guzzle

guzzlehttp/guzzle-services

league/flysystem

phpmailer/phpmailer

symfony/mailer

foooe/router



Komponenten installieren

```
$ composer require vendor/package
```

Guzzle installieren:

```
$ mkdir chucknorrisfact
```

```
$ cd chucknorrisfact
```

```
$ composer require guzzlehttp/guzzle
```

Erzeugt:

`composer.json` und `composer.lock`

`vendor`-Verzeichnis mit Komponenten und Abhängigkeiten

composer.json



```
{  
  "require": {  
    "guzzlehttp/guzzle": "^7.10"  
  }  
}
```

Immer im Projekt belassen (auch im Version Control System).

Komponenten können später einfach installiert werden:

```
$ composer install
```


Versionsangaben



Semantic Versioning:

MAJOR.MINOR.PATCH

z.B. 1.2.4 oder 3.0.8

Wann werden Versionsnummern geändert?

- MAJOR bei inkompatiblen, fundamentalen Änderungen.
- MINOR bei neuen Funktionalitäten (rückwärtskompatibel).
- PATCH bei Bugfixes (rückwärtskompatibel).

Composer Versionierung



```
$ composer require vendor/package:^1.0
```

- Genau: 1.2.3
- Bereich: ≥ 1.0 oder $\geq 1.0 < 2.0$ oder 1.0 - 2.0
- Wildcard: 1.0.* (entspricht $\geq 1.0.0 < 1.1$)
- Nächster Release: ~1.2 (entspricht $\geq 1.2.0 < 2.0.0$),
~1.2.3 (entspricht $\geq 1.2.3 < 1.3.0$)
- Nächster Release (großzügig): ^1.2.3 (entspricht $\geq 1.2.3 < 2.0.0$)

Updates & Autoloading



Pakete updaten:

```
$ composer update
```

Generiert neue composer.lock Datei

Composer Autoloader verwenden:

```
require "vendor/autoload.php";
```

Einbinden in der Haupt-/Index-Datei.

Ein Autoloader passend für alle inkludierten Komponenten.

Beispiel: Kategorien



index.php:
<?php

```
require "vendor/autoload.php";
```

Der Composer Autoloader
lädt alle Packages aus
composer.json

```
$client = new GuzzleHttp\Client();
```

```
$categoriesResponse = $client->request(  
    "GET",  
    "https://api.chucknorris.io/jokes/categories",  
);
```

Ein Guzzlehttp Client wird
angelegt und fragt die
Kategorien per
GET-Request ab.

```
$categoriesBody = $categoriesResponse->getBody();  
$categories = json_decode($categoriesBody);  
?>
```

Beispiel: HTML-Formular



index.php (fortgesetzt):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Random Chuck Norris Fact</title>
  <meta charset="UTF-8">
</head>
<body>
<h1>Get a Random Chuck Norris Fact</h1>
<form action="<?=$_SERVER["SCRIPT_NAME"] ?>" method="post">
  <label for="categorySelector">Select a category for a fact:</label>
  <select name="category" id="categorySelector">
    <?php
      foreach ($categories as $category) {
        echo "<option value=\"\$category\">\"$category</option>\";
      }
    ?>
  </select>
  <button type="submit">Roundhouse!</button>
</form>
```

PHP-Datei ruft sich nach dem Absenden wieder selbst auf.

Übertragung mit POST. Das Formularfeld heißt **category**.

Die Dropdown-Liste wird mit den Kategorien aus der Abfrage befüllt.

...

Beispiel: Fact-Ausgabe



index.php (fortgesetzt):

```
<?php
if (isset($_POST["category"])) {
    echo "<h2>Random fact from category {$_POST["category"]}</h2>";

    $factResponse = $client->request(
        "GET",
        "https://api.chucknorris.io/jokes/random",
        [
            "query" => [
                "category" => $_POST["category"],
            ],
        ],
    );

    $factBody = $factResponse->getBody();
    $fact = json_decode($factBody);
    echo "<p>$fact->value</p>";
}
?>
</body>
</html>
```

Eigene Komponenten



```
$ composer init
```

```
{
  "name": "fhoee/router",
  "description": "A simple object-oriented router for educational purposes.",
  "license": "MIT",
  "type": "library",
  "keywords": [
    "routing",
    "php",
    "education"
  ],
  "authors": [
    {
      "name": "Wolfgang Hochleitner",
      "email": "wolfgang.hochleitner@fh-hagenberg.at",
      "role": "Developer"
    }
  ],
  "require": {
    "php": "^8.5",
    "ext-mbstring": "*",
    "psr/log": "^3.0"
  },
  "require-dev": {
    "ergebnis/composer-normalize": "^2.50",
    "friendsofphp/php-cs-fixer": "^3.94",
    "mockery/mockery": "^1.6",
    "pestphp/pest": "^4.4",
    "phpstan/phpstan": "^2.1"
  },
  "minimum-stability": "stable",
```

```
  "autoload": {
    "psr-4": {
      "Fhoee\\Router\\": "src/"
    }
  },
  "config": {
    "allow-plugins": {
      "ergebnis/composer-normalize": true,
      "pestphp/pest-plugin": true
    }
  },
  "scripts": {
    "pest": "pest",
    "phpstan": "phpstan analyse src --memory-limit=-1 --level 9 || true",
    "test": [
      "cs-check": "php-cs-fixer fix --dry-run --diff",
      "cs-fix": "php-cs-fixer fix",
      "pest": "pest",
      "phpstan": "phpstan analyse src --memory-limit=-1 --level 9 || true",
      "test": [
        "@cs-check",
        "@phpstan",
        "@pest"
      ]
    ]
  }
}
```

Standards in PHP

PHP-FIG, PSRs und PERs

PSR

PHP Standard Recommendations für

Styles | Autoloading | Interfaces

<https://php-fig.org/psr>



PER

PHP Evolving Recommendations

Dinge, die sich rasch ändern (Styles)

<https://www.php-fig.org/per/>



PSR-1 : Basic Coding Stnd.



- PHP-Tags: Nur `<?php ?>` und `<?= ?>`
- Zeichenkodierung: UTF-8 ohne BOM
- Nebeneffekte: Entweder Dinge definieren (Klassen, Funktionen etc.) oder Logik ausführen, aber nie beides. Nebeneffekte sind Ausgaben (`echo`) oder Einbinden mit `require`.
- Autoloading: Alle Namespaces und Klassen folgen einer Autoloading-Strategie (PSR-4)
- Klassennamen: `StudlyCaps`
- Konstantennamen: Großschreibung und Underscores, z.B. `SUM` oder `NR_OF_ELEMENTS`
- Methodennamen: `camelCase()`
- Variablennamen: Entweder `$StudlyCaps`, `$camelCase` oder `$under_score`. Keine direkte Vorschrift, aber innerhalb eines Projektes soll es konsistent sein.

PER: Coding Style



- PSR-1 implementieren.
- Dateien und Zeilenumbrüche: Linux LF, am Ende keine Leerzeile, weiche Zeichenlänge 120 Zeichen, keine überflüssigen Whitespaces. Leerzeilen erlaubt, nur eine Anweisung pro Zeile. Kein schließendes PHP-Tag (`?>`) in Dateien mit nur PHP-Code.
- Einrückungen: 4 Leerzeichen (keine Tabs).
- Schlüsselwörter: Immer klein geschrieben (`if`, `for`, `true`, `false`, `null` etc.). Gilt auch für zukünftige Typen und Schlüsselwörter.
- Reihenfolge von Deklarationen, Namespaces und Importen:
 - Öffnendes `<?php`-Tag.
 - Docblock für die gesamte Datei.
 - `declare`-Angaben (z. B. `declare(strict_types=1);`).
 - Die Namespace-Angabe der Datei (z. B. `namespace vendor\package;`).
 - `use`-Statements für Klassen, Funktionen und/oder Konstanten (z. B. `use Vendor\Package\Class;`).
 - Der Rest der PHP-Datei mit dem Code.

PER: Coding Style



- Abschließendes Komma: Ja, wenn Konstrukte mit Beistrich getrennte Werte mehrzeilig darstellen, z.B.

```
$array = [  
    "wert1",  
    "wert2",  
];
```
- Klassen: Öffnende und schließende Klammern stehen in eigener Zeile, **extends** und **implements** beim Klassennamen.
- Traits: Kommen in die Zeile nach der öffnenden Klammer (z. B. **use MyTrait;**)
- Eigenschaften und Sichtbarkeit: Jede Eigenschaft erhält Sichtbarkeit (**public**, **protected**, **private**). **abstract** und **final** stehen vor der Sichtbarkeit, **static** danach.
- Methoden: Alle Methoden erhalten Sichtbarkeit, die Klammern stehen in eigenen Zeilen, Argumente durch Beistrich getrennt. Beim Aufruf kein Leerzeichen zwischen Methodenname und Klammern (z. B. **\$this→myMethod();**)

PER: Coding Style



- Kontrollstrukturen: Öffnende Klammer steht in derselben Zeile. Weiterführende Strukturen (z. B. `else` oder `elseif`) stehen in der Zeile der schließenden Klammer.
- `elseif` statt `else if`.
- Operatoren: `$i++` oder `++$i` ohne Leerzeichen, sonst mit (z.B. `$result = $a + $b;`).
- Arrays: Immer Kurzschreibweise verwenden: `$array = []`;
- Closures: Leerzeichen nach Schlüsselwort `function`: `$closure = function ($a, $b);`; auch bei Arrow Functions `$func = fn(int $x, int $y) => $x + $y;`
- Enumerations: Wie Klassen, aber nur `public` und `private` erlaubt (keine Vererbung). Fälle in PascalCase oder UPPER_CASE.
- Property Hooks: Mehrzeilig wie Methoden, einzeilig auch möglich.
- Heredoc & Nowdoc: Nowdoc bevorzugt, wenn möglich.
- Heredoc und Nowdoc: Der schließende Identifikator darf eingerückt werden.
- Attribute: In einer eigenen Zeile direkt vor der Struktur.

Autoloading: PSR-0



Voller Pfad aus dem Namespace muss existieren

Klasse: `Acme\Logging\AdvancedLogging`

Verzeichnis: `path/to/project/lib/vendor/Acme/Logging/AdvancedLogging.php`

Klasse: `Fhooe\MTD\Hypermedia`

Verzeichnis: `path/to/project/lib/vendor/Fhooe/MTD/Hypermedia.php`

Unflexibel, daher ist PSR-0 deprecated.

Autoloading: PSR-4



Mapping eines Namespaces auf einen Ordner möglich
Vereinfacht die Ordnerstruktur

Klasse: `Acme\Logging\AdvancedLogging`

Mapping: `Acme` auf `src`

Verzeichnis: `path/to/project/src/Logging/AdvancedLogging.php`

Klasse: `Acme\Logging\AdvancedLogging`

Mapping: `Acme\Logging` auf `src`

Verzeichnis: `path/to/project/src/AdvancedLogging.php`

Composer Autoloader sind PSR-4 kompatibel.



Weitere PSRs

PSR-3 Logger Interface

Standardisiertes Interface für Logger-Klassen. Beispielhaft durch `monolog/monolog` umgesetzt.

PSR-5 PHPDoc-Standard

Definiert die Dokumentation von PHP-Code (noch ein Entwurf).

PSR-6 Caching Interface

Interface für Caching-Komponenten.

PSR-7 HTTP Message Int.

Interface für HTTP Message (z.B. von `Guzzlehttp` für die Response verwendet).

PSR-18 HTTP Client

Interface zum Versenden und Empfangen von HTTP Requests und Responses. Grundlage für Komponenten wie `Guzzlehttp`.

PSR-20 Clock

Beschreibt ein Interface, mit dem die Systemzeit ausgelesen werden kann.

Standards einhalten

PSRs und PERs in der Praxis

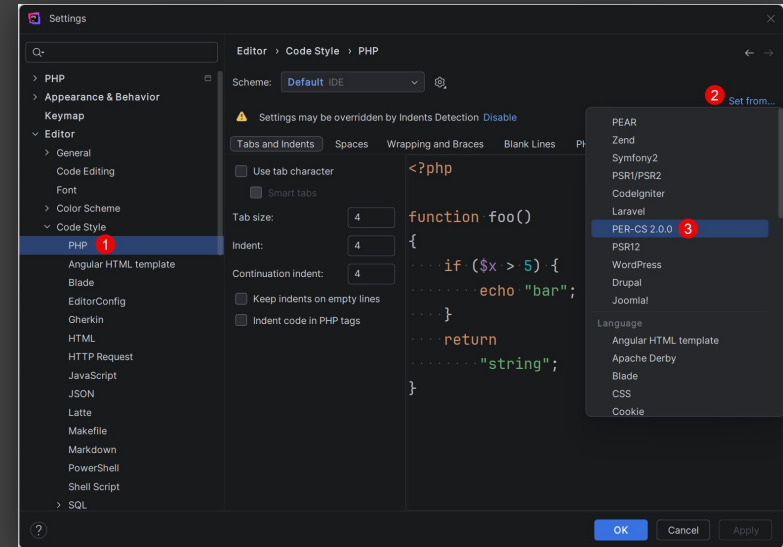


IDE konfigurieren

PhpStorm:

Settings > Editor > Code Style > PHP

Set from... PER-CS 2.0.0



Tools: PHP_CodeSniffer



```
> phpcs --standard=PSR12 /path/to/code/myfile.php
```

Chuck Norris Index-Datei (mit eingebauten Fehlern):

```
> phpcs --standard=PSR12 index_phpcserrors.php
```

Output:

```
FILE: /var/www/html/wet2vo-examples/vo05/chucknorrisfact/index_phpcserrors.php
```

```
-----  
FOUND 2 ERRORS AFFECTING 3 LINES
```

```
-----  
37 | ERROR | [x] Expected 1 space(s) after closing parenthesis; found newline
```

```
53 | ERROR | [x] Line indented incorrectly; expected at least 4 spaces, found 2
```

```
-----  
PHPCBF CAN FIX THE 2 MARKED SNIFF VIOLATIONS AUTOMATICALLY
```

```
-----  
Time: 75ms; Memory: 8MB
```

Reparatur: **phpbcf**



```
> phpbcf --standard=PSR12 /path/to/code/myfile.php
```

Chuck Norris Index-Datei:

```
> phpbcf --standard=PSR12 index.php
```

Output:

PHPCBF RESULT SUMMARY

FILE	FIXED	REMAINING
/.../wet2vo-examples/vo05/chucknorrisfact/index_phpcserrors.php	3	0

A TOTAL OF 2 ERRORS WERE FIXED IN 1 FILE

Time: 103ms; Memory: 8MB

Tools: **PHP-CS-Fixer**



```
> php-cs-fixer check --rules=@PER-CS3x0 /path/to/code/myfile.php
```

Chuck Norris Index-Datei (mit eingebauten Fehlern):

```
> php-cs-fixer check --rules=@PER-CS3x0 --diff index_phpcserrors.php
```

Tools: PHP-CS-Fixer



```
1) index_phpcerrors.php
----- begin diff -----
--- /var/www/html/wet2vo-examples/vo05/chucknorrisfact/index_phpcerrors.php
+++ /var/www/html/wet2vo-examples/vo05/chucknorrisfact/index_phpcerrors.php
@@ -28,14 +28,13 @@
foreach ($categories as $category) {
echo "<option value=\"{$category}\">{$category}</option>";
}
-
-      ?>
+?>
</select>
<button type="submit">Roundhouse!</button>
</form>
<?php
-if (isset($_POST["category"]))
-{
+if (isset($_POST["category"])) {
echo "<h2>Random fact from category {$_POST["category"]}</h2>";
$factResponse = $client->request(
@@ -43,7 +42,7 @@
"https://api.chucknorris.io/jokes/random",
[
"query" => [
-      "category" => $_POST["category"]
+      "category" => $_POST["category"],
],
],
);
----- end diff -----
```

Tools: **PHP-CS-Fixer**



```
> php-cs-fixer fix --rules=@PER-CS3x0 /path/to/code/myfile.php
```

Chuck Norris Index-Datei (mit eingebauten Fehlern):

```
> php-cs-fixer fix --rules=@PER-CS3x0 --diff index_phpcserrors.php
```

Output:

```
1) index_phpcserrors.php
```

```
Fixed 1 of 1 files in 0.021 seconds, 19.41 MB memory used
```




Nächstes Mal in WEF2VO

Templates und Routing

Credits



- S. 3: Foto: <https://www.pexels.com/de-de/foto/person-nagel-hammern-nahaufnahme-6920074/>
- S. 9: Composer Logo: <https://getcomposer.org/>